

Contributing to VendoraPOS

Read this before opening a pull request. It covers what to install on Windows, macOS, or Linux, how to get running locally, and the design rules that keep this codebase consistent.

1. Install these first

Exact versions this project is built and tested against:

Tool	Version used	Target framework / package
.NET SDK	10.0.202	net10.0 (any 10.x SDK should work)
Node.js	24.19.0	
npm	11.17.0	ships with Node
Angular	22.1.x	@angular/core + @angular/cli, installed via npm install — no separate global CLI install needed
dotnet-ef CLI tool	10.0.11	dotnet tool install --global dotnet-ef (same command on every OS)
Docker	any recent version	runs the mcr.microsoft.com/mssql/server:2022-latest image
Git	any recent version	

Windows (PowerShell)

```
winget install Microsoft.DotNet.SDK.10
```

```
winget install OpenJS.NodeJS.LTS      # or use nvm-windows:  
github.com/coreybutler/nvm-windows
```

```
winget install Docker.DockerDesktop  # requires WSL2 backend  
enabled
```

```
winget install Git.Git
```

```
dotnet tool install --global dotnet-ef
```

Docker Desktop on Windows needs the WSL2 backend enabled (Docker Desktop will prompt you to enable it on first run if it isn't already). Everything else — `dotnet`, `npm`, `git` commands — works identically in PowerShell, CMD, or a WSL2 shell.

macOS (with Homebrew)

```
brew install --cask dotnet-sdk
```

```
brew install nvm && nvm install 24 && nvm use 24    # or: brew  
install node@24
```

```
brew install --cask docker                                #  
Docker Desktop
```

```
brew install git
```

```
dotnet tool install --global dotnet-ef
```

Linux (Debian/Ubuntu-based, e.g. Pop!_OS — what this project was built on)

```
# .NET SDK - see learn.microsoft.com/dotnet/core/install/linux  
for your distro's exact repo setup
```

```
sudo apt-get update && sudo apt-get install -y dotnet-sdk-10.0
```

```
# Node via nvm
```

```
curl -o-  
https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.1/install.s  
h | bash
```

```
nvm install 24 && nvm use 24
```

```
# Docker Engine
```

```
sudo apt-get install -y docker.io
```

```
sudo usermod -aG docker $USER    # log out/in after this so  
docker works without sudo
```

```
sudo apt-get install -y git
```

```
dotnet tool install --global dotnet-ef
```

2. First-time setup

Step 1 — Clone and start SQL Server

Matches the connection string already committed in `src/server/Vendora.Api/appsettings.json` — this is a local dev-only password, not a real secret. This `docker run` command is identical on Windows (PowerShell/CMD), macOS, and Linux:

```
docker run -e "ACCEPT_EULA=Y" -e  
"SA_PASSWORD=YourStrong@Passw0rd" \  
  
-p 1433:1433 --name eca-mssql -d  
mcr.microsoft.com/mssql/server:2022-latest
```

If you use a different container name/password, update

`ConnectionStrings:DefaultConnection` in `appsettings.json` locally — just don't commit real production credentials there (see `.gitignore` — `appsettings.Production.json` is already excluded for that reason).

Step 2 — Run the API (no manual migration step needed)

```
./scripts/run-api.sh          # dotnet run

./scripts/run-api.sh watch    # dotnet watch run (auto-reload)
```

On startup, in the Development environment, the API automatically applies any pending EF Core migrations (creating the database schema from nothing, if needed) and then seeds 20 sample products (spanning pharmacy/retail/coffee-shop, matching the generic data model) — but only if the Products table is completely empty, so it never touches or duplicates data on a database you've already been working in. See `Program.cs` and `Vendora.Infrastructure/Persistence/DbSeeder.cs`.

`./scripts/run-api.sh` frees ports 7196/5136 first, so re-running is always safe. It's a bash script — works as-is in a Linux/macOS terminal, or in **Git Bash** or **WSL2** on Windows. If you're in native PowerShell/CMD, just run `dotnet run --launch-profile https` directly instead (from `src/server/Vendora.Api`); if you hit a "port already in use" error, find and stop the process with:

```
Get-Process -Id (Get-NetTCPConnection -LocalPort
7196).OwningProcess | Stop-Process
```

Either way, you must pass `--launch-profile https` when running manually, or the Angular proxy won't be able to reach the API — see the README.

If you ever need to run migrations manually instead (e.g. against a database the API isn't

pointed at), from `src/server/Vendora.Api`:

```
dotnet ef database update --project ../Vendora.Infrastructure
--startup-project .
```

Step 3 — Run the client

From `src/client`:

```
npm install
```

```
npm start
```

Serves on `http://localhost:4200`, proxying `/api/*` and `/uploads/*` to the API (see `proxy.conf.json`). **If you edit `proxy.conf.json`, stop and re-run `npm start`** — Angular's dev server only reads that file once at startup, so a running instance won't pick up new proxy rules until it's restarted.

See `README.md` and `docs/architecture.md` for more detail.

3. Rules that keep the design intact

This is a small, deliberately consistent codebase. Please follow the existing patterns rather than introducing new ones — if you think a pattern needs to change, raise it in the PR description first.

Backend — Clean Architecture, strictly one-directional

- `Vendora.Domain` has **zero** dependencies — no EF Core, no ASP.NET Core types. Ever.

- `Vendora.Application` depends on Domain only. No EF Core here either (`DbUpdateException` etc. stay in Infrastructure).
- `Vendora.Infrastructure` depends on Application + Domain; this is the only layer allowed to know about EF Core.
- `Vendora.Api` depends on Application + Infrastructure; controllers stay thin — one action per use case, translate results to HTTP status codes, no business logic.
- Entities: private constructor + a static `Create(...)` factory that enforces invariants. Behavior lives on the entity as methods (`AdjustStock`, `Deactivate`, `UpdateDetails`) — never public setters, never business logic in a service that could live on the entity.
- Immutable ledger entities (`StockMovement`, `ProductAuditLog`) never get an Update/mutation method — they're an append-only audit trail. Don't add editing to these; if a past entry is wrong, the pattern is a compensating entry (see the "Reverse" flow), not editing history.
- One repository per aggregate root, implementing the generic `IRepository<T>`. Repositories sharing a `DbContext` can flush each other's pending changes via a single `SaveChangesAsync()` call — you don't need a separate call per repository.

Frontend — Angular, standalone components

- No `NgModules`. Standalone components only, matching what's already there.
- Local component state via `signal()`, not manual fields + `ChangeDetectorRef`.
- Reactive forms (`FormBuilder`) for anything with validation.
- Use `inject()` for dependencies in components that also build a form field with `fb.group(...)` — field initializers run before constructor-injected params are assigned, so constructor injection breaks there. Plain services (`ProductService`, etc.) still use constructor injection — match whichever file you're editing.
- **Reuse the existing design-system classes** in `src/client/src/styles.scss` (`.btn/.btn-primary/.btn-danger/.btn-outline`, `.field`, `.responsive-table`, `.error/.error-banner/.muted`) instead of writing new one-off CSS for buttons, forms, or tables.
- Anything with a table must work on mobile — use the `.responsive-table` class + `data-label` attributes on `<td>s` (see `product-list.html` or `product-stock-history.html` for the pattern), and check it at <640px width before calling it done.

- **Every list/table page uses the fixed-viewport `.page-fixed` layout** — the page itself never grows past the visible screen or produces its own page-level scrollbar, no matter how much data it holds. Add `class="page-fixed"` to the page's root element, and wrap exactly one child (usually the table) in `<div class="page-scroll">` — everything else (heading, search/toolbar, pagination) stays fixed in place while only that wrapped part scrolls internally. See `product-list.html/sale-history.html` for the pattern (defined once in `styles.scss`, relaxes to normal full-page scroll under 640px since a cramped internal scroll region doesn't feel natural on a phone). Checkout uses a bespoke variant of the same idea (`checkout.scss`) since it has two independently-scrolling panes instead of one (product grid, current-transaction cart) — same principle, applied twice, and deliberately *not* relaxed on phones like `.page-fixed` is, since the transaction panel needs to stay visible at all times regardless of screen size.

Don't speculate ahead of the current module

No `Sale` stock-movement reason, no `TenantId`, no auth-related fields — those belong to modules that don't exist yet (see README's Status list) and should be designed when that module is actually built, not guessed at now.

Docs

Every user-facing feature gets a new section in `docs/user-guide.md` **and** `src/client/src/app/features/help/help.html` (the in-app Help page) **and** `docs/user-guide.html` (source for the PDF/Word exports in `docs/exports/`) — all three need to stay in sync.

Before opening a PR

- `dotnet build` and `cd src/client && npx ng build` both need to pass cleanly.
- Click through the feature manually at least once — type checking isn't the same as it actually working.
- `master` is protected — you can't push directly to it. Branch, push, open a PR, and wait for review/approval.